

RiskGate

Documentación técnica completa del proyecto
y recomendaciones de evolución

Proyecto RISKGATE — MVP local-first

Mayo de 2026

Resumen

RISKGATE es una aplicación *local-first* de gestión de portafolio y de riesgo en opciones. Su función no es ejecutar órdenes, predecir precios ni dar asesoría financiera, sino **imponer disciplina de proceso** antes de entrar a una operación: obliga a documentar cada idea de trade, la evalúa con un modelo cuantitativo transparente, rechaza estructuras incompletas o ilíquidas y asigna un tamaño de riesgo conservador acotado por los límites de la cuenta. Este documento describe de forma exhaustiva la arquitectura, el modelo de datos, las pantallas, el motor de evaluación (puntaje ponderado adaptativo, calidad de opciones continua, indicadores técnicos, analítica probabilística, simulación Monte Carlo, sentimiento de Reddit y concentración sectorial), el bot de investigación en Python y un conjunto detallado de recomendaciones y modificaciones de *proxies* para hacer el sistema más robusto.

Índice

1. Propósito y filosofía del sistema	4
1.1. Principios de seguridad (local-first)	4
2. Arquitectura y stack tecnológico	4
3. Modelo de datos	5
3.1. Aislamiento por portafolio	5
4. Módulos y pantallas de la aplicación	6
4.1. Dashboard	6
4.2. Portafolio	6
4.3. Importación de Excel de GBM	6
4.4. Watchlist	6
4.5. Discover	6
4.6. Options Chain	6
4.7. Trade Ideas	6
4.8. Risk Model	7
4.9. Model Guide	7
4.10. Journal	7
4.11. Settings	7

5. Pipeline de datos de mercado	7
5.1. Yahoo Finance (<code>lib/market-data.ts</code>)	7
5.2. Alpaca Options Data API (<code>lib/alpaca-options.ts</code>)	7
6. Motor de evaluación de ideas de trade	8
6.1. Estructura general	8
6.2. Pesos adaptativos al régimen	8
6.3. Puntaje de régimen de mercado	8
6.4. Puntaje técnico	9
6.5. Puntaje de calidad de opciones (continuo)	9
6.6. Puntaje de calidad del trade (continuo)	9
6.7. Capas de superposición	9
7. Analítica de opciones	10
7.1. Probabilidad de ganancia (PoP)	10
7.2. Valor esperado y Kelly	10
7.3. VaR y CVaR (un día)	10
7.4. Greeks en dólares y agregación	10
7.5. Estructuras multi-leg (<code>lib/options/multileg.ts</code>)	11
8. Simulación Monte Carlo	11
9. Sentimiento de Reddit (opcional)	11
10. Concentración sectorial	11
11. Decisión y dimensionamiento de riesgo	12
11.1. Bandas de decisión	12
11.2. Hard stops (rechazo forzado)	12
11.3. Topes de aprobación (limitan a watchlist)	12
11.4. Fórmula de dimensionamiento	12
11.5. Límites por defecto (cuenta de 100,000 MXN)	13
12. Descubrimiento e ideas automáticas	13
12.1. Descubrimiento (<code>lib/discovery.ts</code>)	13
12.2. Ideas automáticas (<code>lib/auto-ideas.ts</code>)	13
12.3. Selección de contrato (<code>lib/option-selector.ts</code>)	13
12.4. Refresco diario (<code>lib/daily-refresh.ts</code>)	13
13. Métricas de la bitácora	14
14. Bot de investigación en Python	14
14.1. Black–Scholes y proxy de IV	14
14.2. Scripts de validación	15
15. Limitaciones actuales	15

16.Recomendaciones y próximos pasos	15
16.1. Corto plazo: robustez de datos y <i>proxies</i>	15
16.2. Mediano plazo: cobertura del modelo	16
16.3. Largo plazo: producto y plataforma	16
16.4. Candidatos de integración externa	16
17.Cómo ejecutar	17
17.1. Aplicación web	17
17.2. Bot de investigación	17
18.Notas de seguridad	17
19.Conclusión	17

1. Propósito y filosofía del sistema

RISKGATE fue diseñado como un sistema de apoyo a la decisión para trading manual de acciones y opciones. La regla central que gobierna todo el diseño es:

Ninguna operación de opciones debe aprobarse a menos que el plan escrito, el puntaje ponderado, los controles de liquidez, las reglas de vencimiento y los topes de riesgo del portafolio se cumplan simultáneamente.

Los objetivos concretos del sistema son:

- mantener varios portafolios (cuentas) aislados entre sí;
- importar datos de portafolio exportados manualmente desde GBM;
- registrar posiciones, efectivo, exposición y asignación de activos;
- mantener una *watchlist* de tickers seguidos por portafolio;
- obligar a que cada idea de opciones tenga tesis escrita, nivel de invalidación, pérdida máxima y plan de salida;
- puntuar ideas combinando contexto de mercado, técnico, calidad del contrato y calidad del plan;
- rechazar configuraciones incompletas o inseguras;
- calcular un tamaño de riesgo sugerido acotado por los límites de cuenta; y
- registrar operaciones entradas y cerradas en una bitácora para revisión.

1.1. Principios de seguridad (local-first)

El diseño local-first se eligió porque la aplicación maneja información financiera. El MVP evita deliberadamente:

- almacenar contraseñas de GBM o del broker;
- hacer *scraping* de páginas de inicio de sesión;
- sincronizar datos privados a la nube;
- trading automático o colocación de órdenes.

La base de datos SQLite vive en la máquina del usuario y todas las fuentes externas se consumen en modo *solo lectura*.

2. Arquitectura y stack tecnológico

El producto se compone de dos piezas independientes:

1. **La aplicación web** (Next.js / React / TypeScript), que es el MVP principal y contiene el motor de evaluación.
2. **El bot de investigación** (options-strategy-bot, Python), un banco de pruebas independiente para escanear, puntuar, hacer *backtest* y llevar un libro de *paper trading*.

Capa	Implementación
Frontend	Next.js (App Router), React, TypeScript
Estilos	Tailwind CSS, @tailwindcss/forms

Lógica de servidor	Server Actions de Next.js (<code>app/actions.ts</code>)
Base de datos	SQLite
ORM	Prisma Client (<code>@prisma/client v6</code>)
Gráficas	Recharts
Iconos	Lucide React
Validación	Zod
Importación Excel	paquete <code>read-excel-file</code>
Datos de mercado	Yahoo Finance (endpoint público de <i>chart</i>)
Cadena de opciones	Alpaca Options Data API (solo lectura, opcional)
Bot de investigación	Python 3, Black-Scholes propio, Yahoo/Alpaca/CSV
Runtime	Servidor de desarrollo local de Node.js

Todas las páginas se sirven con `dynamic = "force-dynamic"` para reflejar datos en tiempo real. Cada operación de mutación se ejecuta como *server action* y revalida las rutas afectadas tras escribir en la base de datos.

3. Modelo de datos

El esquema Prisma (SQLite) organiza la información alrededor de la entidad **Portfolio**, que aísla cuentas. Las entidades principales son:

Modelo	Propósito
Portfolio	Cuenta o portafolio independiente; ancla del aislamiento de datos.
Position	Tenencias manuales o importadas (acción, ETF, opción, efectivo).
WatchlistItem	Tickers seguidos con cotización, cambio diario y fuente.
TradeIdea	Idea de opciones pre-entrada: inputs, puntajes, decisión, riesgo sugerido, razones y advertencias.
OptionLeg	Pierna individual de una estructura multi-leg (tipo, lado, strike, vencimiento, Greeks, IV, OI, volumen).
ModelScore	Registro normalizado de puntaje, uno a uno con la idea.
TradeScoreSnapshot	Foto diaria del puntaje de una idea para seguir su evolución.
RiskSettings	Límites de riesgo y parámetros de dimensionamiento por portafolio.
TradeJournal	Operaciones entradas y cerradas, ligadas a una idea.
PriceBar	Barras OHLCV históricas por símbolo y timeframe.
PortfolioSnapshot	Foto puntual del valor del portafolio (intradía e histórico).
RedditDocument	Documentos de Reddit normalizados para análisis de sentimiento.

3.1. Aislamiento por portafolio

Los modelos **Position**, **TradeIdea**, **RiskSettings**, **WatchlistItem**, **PriceBar** y **PortfolioSnapshot** llevan un campo `portfolioId`. El portafolio activo se transmite por la URL:

`?portfolio=<portfolioId>`

Las entradas de bitácora se ligan a través de la idea de trade, por lo que están indirectamente aisladas por portafolio.

4. Módulos y pantallas de la aplicación

La barra lateral (`components/sidebar.tsx`) contiene el selector de portafolio y la navegación a diez pantallas.

4.1. Dashboard

Resume el portafolio seleccionado: equity total, efectivo, exposición a acciones/ETF y opciones, cambio intradía, riesgo de opciones utilizado, tope de pérdida mensual, gráfica de asignación (pastel), curva de equity, desempeño intradía, gráfica de uso de riesgo y la tabla de ideas recientes con su decisión.

4.2. Portafolio

Permite crear, editar y eliminar posiciones e importar el Excel de GBM. Cada posición tiene símbolo, tipo de activo, mercado (MX/US), cantidad, costo promedio, precio actual, moneda, cuenta y notas. Refresca precios desde Yahoo y registra *snapshots* para el desempeño intradía.

4.3. Importación de Excel de GBM

El importador lee el libro de detalle de GBM, ignora encabezados de sección (p.ej. **Mercado de Capitales USA**, **Liquidez**), interpreta filas de tickers, convierte **efectivo** en una posición **CASH**, reconoce ETFs comunes y, opcionalmente, convierte valores USD a MXN con el tipo de cambio ingresado. Puede reemplazar todas las posiciones o fusionar por símbolo.

4.4. Watchlist

Almacena tickers seguidos por portafolio (símbolo, nombre, mercado, moneda, fuente, precio, cierre previo, cambio diario, marca de tiempo). Permite refrescar cotizaciones desde Yahoo o editarlas manualmente. Es la base para el cálculo automático de indicadores técnicos.

4.5. Discover

Pantalla de descubrimiento: escanea el mercado y la watchlist para encontrar candidatos. Filtros por perfil (*quality* o *movers*), puntaje mínimo, volumen mínimo e inclusión de Reddit. Muestra puntaje de descubrimiento, sesgo (alcista/bajista), volatilidad realizada, menciones en Reddit y probabilidad de liquidez de opciones, con botones para añadir a watchlist o crear una idea.

4.6. Options Chain

Navega la cadena de opciones de Alpaca por subyacente, vencimiento y strike. Muestra bid/ask/mid, delta, gamma, theta y vega, y permite pre-llenar el formulario de idea con el contrato elegido.

4.7. Trade Ideas

Es el *núcleo de la compuerta de riesgo*. Cada posible operación se escribe aquí antes de entrar. El formulario captura símbolo, dirección, estrategia, tesis, catalizador, nivel de invalidación, objetivo, vencimiento, prima, pérdida máxima, recompensa esperada, plan de salida, estructura multi-leg, contexto de mercado, señales técnicas e indicador de *chasing*. Por cada idea muestra los cuatro

puntajes de módulo, la decisión, el riesgo sugerido, la analítica de opciones (PoP, EV, Kelly, VaR), el histograma Monte Carlo, el diagrama de payoff, el desglose técnico y de calidad de opciones, el sentimiento de Reddit y las listas de razones y advertencias.

4.8. Risk Model

Tablero del perfil de riesgo: riesgo base por trade, límite del *sleeve* de opciones, pesos por régimen, bandas de decisión, multiplicadores, Greeks agregados del portafolio y analítica de concentración.

4.9. Model Guide

Explicación didáctica del modelo dentro de la app: módulos de puntaje, hard stops, topes de aprobación, bandas de decisión, fórmula de dimensionamiento, metodología de sentimiento y hoja de ruta de robustez.

4.10. Journal

Bitácora de operaciones: enlaza la idea, fechas y precios de entrada/salida, P&L realizado, resultado, etiquetas de error y notas. Calcula *win rate*, ganancia/pérdida promedio, *expectancy*, proxies de Sharpe y Sortino, duración media e intervalos de confianza por *bootstrap*.

4.11. Settings

Gestión de portafolios (crear/editar/eliminar) y de los límites de riesgo: valor total, *sleeve* de opciones, riesgo base por trade, tope de pérdida mensual, tope de riesgo abierto y la configuración de sentimiento de Reddit.

5. Pipeline de datos de mercado

5.1. Yahoo Finance (`lib/market-data.ts`)

Obtiene barras OHLCV (por defecto 6 meses diarios) y cotizaciones. A partir del historial construye un *resumen técnico* por símbolo y el `MarketContext` agregado:

- **Tendencia de régimen:** *uptrend/downtrend/neutral* según precio vs. SMA20/SMA50 y cambio a 20 días.
- **Condición de VIX:** *low* (< 15), *normal*, *elevated* (22–30), *spiking* (≥ 30 o salto diario $\geq +15\%$).
- **Amplitud (breadth):** proporción de símbolos con precio $\geq$ SMA50. *Strong* si $\geq 65\%$, *weak* si $\leq 40\%$.
- **Riesgo macro:** *high* si el VIX está *spiking*; *low* si VIX bajo y breadth fuerte; *medium* en otro caso.

5.2. Alpaca Options Data API (`lib/alpaca-options.ts`)

Consulta *snapshots* de la cadena de opciones (Greeks, IV, bid/ask, volumen, open interest) en modo solo lectura. Decodifica el símbolo OCC [ROOT] [YY] [MM] [DD] [C/P] [STRIKE $\times$ 1000] y normaliza respuestas en múltiples formatos. Requiere `APCA_API_KEY_ID` y `APCA_API_SECRET_KEY`; si no están configuradas, la app opera con datos manuales.

6. Motor de evaluación de ideas de trade

El corazón del sistema es `evaluateTradeIdea()` en `lib/risk.ts`. A diferencia de versiones anteriores con ajustes discretos, el modelo actual usa **funciones de calidad continuas, pesos adaptativos al régimen y capas de superposición** probabilísticas.

6.1. Estructura general

El puntaje total se compone de cuatro módulos en escala $[0, 100]$:

Módulo	Peso típico	Propósito
Régimen de mercado (M)	25–35 %	Contexto amplio: SPY, QQQ, VIX, breadth, macro.
Técnico (T)	20–30 %	Alineación direccional con tendencia, medias, RSI, volumen, S/R.
Calidad de opciones (O)	25–35 %	Liquidez y estructura del contrato.
Calidad del trade (Q)	15–20 %	Disciplina del plan escrito.

La función de recorte es:

$$\text{clamp}(x, 0, 100) = \min(100, \max(0, x)). \quad (1)$$

6.2. Pesos adaptativos al régimen

Los pesos w se ajustan según el contexto (`getAdaptiveWeights`):

Régimen	w_M	w_T	w_O	w_Q
VIX elevado/spiking o SPY bajista	0.30	0.20	0.35	0.15
SPY alcista	0.25	0.30	0.25	0.20
Neutral (por defecto)	0.35	0.25	0.25	0.15

El puntaje base ponderado es:

$$S_{\text{base}} = w_M M + w_T T + w_O O + w_Q Q. \quad (2)$$

6.3. Puntaje de régimen de mercado

Parte de 50 y aplica ajustes discretos:

$$\begin{aligned}
 M = \text{clamp} \big(& 50 + 15\mathbb{I}\{\text{SPY alcista}\} - 15\mathbb{I}\{\text{SPY bajista}\} \\
 & + 15\mathbb{I}\{\text{QQQ alcista}\} - 15\mathbb{I}\{\text{QQQ bajista}\} \\
 & + 10\mathbb{I}\{\text{breadth fuerte}\} - 10\mathbb{I}\{\text{breadth débil}\} \\
 & + 5\mathbb{I}\{\text{VIX bajo}\} - 10\mathbb{I}\{\text{VIX elevado}\} - 25\mathbb{I}\{\text{VIX spiking}\} \\
 & + 5\mathbb{I}\{\text{macro bajo}\} - 20\mathbb{I}\{\text{macro alto}\}, 0, 100 \big). \quad (3)
 \end{aligned}$$

6.4. Puntaje técnico

Cuando hay barras OHLCV disponibles, el técnico se calcula con indicadores reales (`lib/indicators.ts`) en lugar de etiquetas manuales:

- **SMA_n**: media simple de los últimos n cierres.
- **EMA_n**: media exponencial con multiplicador $\alpha = 2/(n + 1)$.
- **RSI₁₄**: $RSI = 100 - \frac{100}{1 + RS}$, $RS = \frac{\overline{\text{ganancia}}}{\overline{\text{pérdida}}}$.
- **ATR₁₄**: media del *true range* $\max(H - L, |H - C_{-1}|, |L - C_{-1}|)$.
- **Volumen relativo**: volumen actual / promedio base.
- **BWAP**: $\sum(p_{\text{típico}} \cdot V) / \sum V$, con $p_{\text{típico}} = (H + L + C)/3$.

El puntaje técnico continuo pondera siete sub-calidades: trend (25 %), alineación de medias (20 %), RSI (15 %), volumen (15 %), BWAP (10 %), soporte/resistencia (10 %) y riesgo por ATR (5 %). Si no hay snapshot calculado, el sistema recurre a un modelo de respaldo con ajustes discretos sobre una base de 50.

6.5. Puntaje de calidad de opciones (continuo)

En `lib/risk/optionsQuality.ts`, cada dimensión usa una función suave en $[0, 1]$ que luego se pondera a $[0, 100]$:

$$\text{spread}(s) = \frac{1}{1 + \left(\frac{s-2}{5}\right)^3}, \quad s > 2\%, \quad (4)$$

$$\text{volumen}(v) = \text{clamp}\left(\sqrt{v/500}\right), \quad (5)$$

$$\text{OI}(oi) = \text{clamp}\left(\sqrt{oi/1000}\right), \quad (6)$$

$$\text{ivRank}(r) = 1 - (r/100)^2, \quad (7)$$

$$\text{theta}(\theta, m) = 1 - |\theta/m|, \quad (8)$$

y la calidad de DTE tiene su máximo en 21–60 días y cae a cero por debajo de 5 o por encima de 240 días. Los pesos son: spread 30 %, DTE 20 %, volumen 15 %, open interest 15 %, IV rank 8 %, tamaño de prima 7 % y theta 5 %.

6.6. Puntaje de calidad del trade (continuo)

$$Q = 100(0.25 q_{\text{tesis}} + 0.10 q_{\text{cat}} + 0.15 q_{\text{inval}} + 0.15 q_{\text{salida}} + 0.20 q_R + 0.15 q_{\text{chase}}), \quad (9)$$

donde $q_{\text{tesis}} = \text{clamp}(\text{long}/80)$ (longitud del texto), $q_{\text{cat}} = 1$ si hay catalizador (si no, 0.35), $q_{\text{inval}} = 1$ si existe nivel de invalidación, $q_{\text{salida}} = \text{clamp}(\text{long}/80)$, $q_R = \text{clamp}(R/3)$ con $R = \text{recompensa/pérdida máxima}$, y $q_{\text{chase}} = 0$ si la idea está marcada como *chasing*.

6.7. Capas de superposición

Sobre S_{base} se aplican ajustes aditivos:

- **Monte Carlo:** +5 si $EV > 0$ y $PoP > 55\%$; -5 si $EV < 0$; -10 si $CVaR_{95} \geq 0.85 \cdot$ pérdida máxima.
- **Sentimiento de Reddit** (si está activo): +3 si concuerda con la dirección y la confianza > 0.65 ; -3 si se opone; penalizaciones por *hype* o por preocupaciones severas.

$$S_{\text{total}} = \text{clamp}(S_{\text{base}} + \Delta_{\text{MC}} + \Delta_{\text{Reddit}}, 0, 100). \quad (10)$$

7. Analítica de opciones

El módulo `lib/options-math.ts` provee analítica probabilística.

7.1. Probabilidad de ganancia (PoP)

Bajo un modelo lognormal del subyacente, para un strike K , precio S_0 , volatilidad implícita σ y tiempo a vencimiento τ (en años):

$$d = \frac{\ln(K/S_0) + \frac{1}{2}\sigma^2\tau}{\sigma\sqrt{\tau}}, \quad PoP_{\text{call}} = 1 - N(d), \quad PoP_{\text{put}} = N(d), \quad (11)$$

donde $N(\cdot)$ es la CDF normal estándar (vía función de error). El punto de equilibrio es $K +$ prima (call) o $K -$ prima (put).

7.2. Valor esperado y Kelly

$$EV = PoP \cdot \text{recompensa} - (1 - PoP) \cdot \text{pérdida máxima}, \quad (12)$$

$$f^* = \max\left(0, \frac{PoP \cdot b - (1 - PoP)}{b}\right), \quad (13)$$

con b el cociente recompensa/riesgo. El sistema usa **Kelly fraccional al 25%** ($f_{1/4} = 0.25 f^*$) como tope de tamaño.

7.3. VaR y CVaR (un día)

Usando una aproximación delta-gamma-vega más decaimiento por theta:

$$\Delta S = S_0 \cdot \frac{\sigma}{\sqrt{252}}, \quad (14)$$

$$\sigma_{\text{proxy}} = |\delta| \Delta S + \frac{1}{2} |\gamma| \Delta S^2 + |\nu| \sigma \cdot 0.1, \quad (15)$$

$$VaR_{95} = \sigma_{\text{proxy}} z_{95} + \theta_{\text{loss}}, \quad z_{95} = 1.645, \quad (16)$$

$$CVaR_{95} = \sigma_{\text{proxy}} \cdot \frac{\phi(z_{95})}{0.05} + \theta_{\text{loss}}. \quad (17)$$

7.4. Greeks en dólares y agregación

La exposición se escala por 100 acciones por contrato. La función `aggregateOptionAnalytics` suma delta, gamma, theta y vega de todas las ideas aprobadas/entradas para mostrar los Greeks del portafolio.

7.5. Estructuras multi-leg (`lib/options/multileg.ts`)

El payoff al vencimiento por pierna es $\max(S_T - K, 0)$ (call) o $\max(K - S_T, 0)$ (put). El P&L de una pierna long es $q \cdot 100 \cdot (\text{payoff} - \text{prima})$ y se invierte para piernas short. La calculadora evalúa el payoff en una malla de puntos (101 regulares más los strikes y el precio actual), localiza los puntos de equilibrio por interpolación lineal y deriva débito/crédito neto, pérdida y ganancia máximas, y Greeks netos. Existen plantillas para *long call/put*, *bull call* y *bear put debit spreads*, *protective put* y *collar*, y un detector de piernas short desnudas que actúa como hard stop.

8. Simulación Monte Carlo

En `lib/options/monteCarlo.ts` se simulan trayectorias del subyacente con un *movimiento browniano geométrico* (GBM):

$$S_T = S_0 \exp\left[\left(\mu - q - \frac{1}{2}\sigma^2\right)\tau + \sigma\sqrt{\tau}Z\right], \quad Z \sim \mathcal{N}(0, 1), \quad (18)$$

con deriva μ (por defecto 0) y rendimiento por dividendos q . Para cada trayectoria se valúa la estructura multi-leg y se construye la distribución de P&L, de la que se obtienen PoP, valor esperado, mediana, percentiles P5/P95, CVaR₉₅ y un histograma. El motor usa una semilla determinista (función del símbolo y el puntaje) para reproducibilidad, con miles de trayectorias por evaluación.

9. Sentimiento de Reddit (opcional)

El subsistema `lib/sentiment/reddit/` resume narrativa pública como señal cualitativa, siempre opcional y desactivada por defecto.

- **Recolección:** busca en Reddit vía OAuth posts sobre el ticker y sus catalizadores, ponderando subreddits (options 1.0, investing/stocks 0.9, wallstreetbets 0.65).
- **Limpieza:** elimina URLs y menciones, decodifica HTML, colapsa espacios y tokeniza.
- **Embeddings TF-IDF locales:** sin modelos externos; se calcula similitud por coseno $\cos(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$.
- **Recuperación:** puntaje = 0.65 coseno + 0.20 recencia + 0.10 engagement + 0.05 peso de subreddit.
- **Clasificación:** produce etiquetas (*strong_bullish*, *bullish*, *mixed*, *bearish*, *strong_bearish*, *hype*, *fear*, *low_signal*) a partir de puntajes de sentimiento, hype y miedo, junto con argumentos, preocupaciones recurrentes y catalizadores mencionados.

El sentimiento ajusta tanto el puntaje (capa de superposición) como un multiplicador de riesgo ($0.75\times$ ante hype alto o desacuerdo con alta confianza).

10. Concentración sectorial

En `lib/concentration.ts` se mide la concentración con el índice de Herfindahl–Hirschman:

$$\text{HHI} = \sum_k w_k^2, \quad w_k = \frac{\text{riesgo}_k}{\text{riesgo total}}, \quad (19)$$

donde k recorre sectores. El multiplicador de penalización es 0.5 si $\text{HHI} > 0.5$ (concentración alta), 0.75 si $\text{HHI} > 0.35$ (moderada) y 1.0 en otro caso. Este multiplicador entra directamente en el dimensionamiento.

11. Decisión y dimensionamiento de riesgo

11.1. Bandas de decisión

Puntaje total	Decisión inicial
$S < 60$	reject
$60 \leq S < 75$	watchlist
$75 \leq S < 85$	approved_small
$S \geq 85$	approved_normal

11.2. Hard stops (rechazo forzado)

Sin importar el puntaje, cualquiera de estos fuerza *reject*:

- falta tesis escrita, nivel de invalidación, pérdida máxima o plan de salida;
- spread bid/ask > 15 %; DTE < 7; prima > 1 % del portafolio;
- volumen < 25 y open interest < 100 simultáneamente;
- datos de pierna faltantes que impiden calcular la pérdida máxima;
- estrategia de riesgo definido sin pérdida máxima determinable;
- piernas short desnudas.

11.3. Topes de aprobación (limitan a watchlist)

La idea no puede superar *watchlist* si: está marcada como *chasing*, IV rank > 80, IV del contrato > 85 %, theta > 8 % del mid por día, o gamma alto con DTE < 30.

11.4. Fórmula de dimensionamiento

El riesgo por multiplicadores es:

$$R_{\text{mult}} = B \cdot C(S) \cdot L(s) \cdot G \cdot V(r) \cdot K_{\text{conc}} \cdot K_{\text{MC}} \cdot K_{\text{sent}}, \quad (20)$$

con B el riesgo base por trade (MXN), y los multiplicadores:

Multiplicador	Regla
Confianza $C(S)$	0 si $S < 75$; 0.25 (75–80); 0.50 (80–85); 0.75 (85–90); 1.0 (≥ 90).
Liquidez $L(s)$	0 si spread > 15 %; 0.25 (> 10 %); 0.50 (> 5 %); 1.0 (≤ 5 %).
Régimen G	1.0 si alineado; 0.50 si neutral; 0.25 si se opone (proxy: tendencia de SPY).
Volatilidad $V(r)$	1.0 si falta IV rank; 0.25 (> 80); 0.50 (60–80); 0.75 (< 20).
Concentración K_{conc}	1.0 a 0.5 según HHI.
Monte Carlo K_{MC}	0.5–1.0 según CVaR/EV/PoP.
Sentimiento K_{sent}	0.75 ante hype alto o desacuerdo confiado; 1.0 en otro caso.

Luego se aplica el **tope de Kelly** y los topes del portafolio:

$$R_{\text{uncapped}} = \min(R_{\text{mult}}, R_{\text{Kelly}}), \quad (21)$$

$$\text{SuggestedRisk} = \begin{cases} \min(R_{\text{uncapped}}, \text{sleeve}_{\text{rest}}, \text{mensual}_{\text{rest}}, \text{abierto}_{\text{rest}}), & \text{si aprobada,} \\ 0, & \text{en otro caso.} \end{cases} \quad (22)$$

11.5. Límites por defecto (cuenta de 100,000 MXN)

Parámetro	Valor por defecto
Valor total del portafolio	100,000 MXN
Sleeve de opciones (3 %)	3,000 MXN
Riesgo base por trade (0.5 %)	500 MXN
Pérdida mensual máxima (1 %)	1000 MXN
Riesgo abierto máximo (2 %)	2,000 MXN

12. Descubrimiento e ideas automáticas

12.1. Descubrimiento (`lib/discovery.ts`)

Combina varias fuentes (Yahoo *day_gainers/losers/most_actives*, un universo líquido de megacaps y menciones de Reddit) y puntúa cada candidato con componentes de configuración técnica, volatilidad realizada, fuerza relativa, ratio de volumen y señal social, aplicando una **penalización por extensión** cuando el movimiento intradía es grande (para no perseguir). Hay dos perfiles de pesos: *quality* (configuraciones limpias) y *movers* (reactivo a momentum y volumen).

12.2. Ideas automáticas (`lib/auto-ideas.ts`)

`ensureDailyAutoIdeas` genera hasta cierto número de ideas por día: descubre candidatos, arma el contexto de mercado, selecciona el mejor contrato de opción vía Alpaca (`lib/option-selector.ts`), construye el `TradeInput`, lo puntúa y lo guarda como *watchlist* o *reject* (nunca aprobada automáticamente, conservando la aprobación manual).

12.3. Selección de contrato (`lib/option-selector.ts`)

Puntúa cada contrato de la cadena con una rúbrica: ajuste de delta (22 %, objetivo 0.40), calidad de spread (20 %), liquidez (18 %, volumen + OI), ajuste de DTE (14 %, máximo en ~ 42 días), IV (10 %), theta (8 %), gamma (4 %) y *moneyness* (4 %), con un umbral mínimo de 48/100.

12.4. Refresco diario (`lib/daily-refresh.ts`)

Orquesta la actualización: refresca cotizaciones de la watchlist, obtiene el historial de SPY/QQ-Q/VIX, reconstruye el contexto de mercado, actualiza técnicos y snapshots de Alpaca por idea, re-puntúa y guarda un `TradeScoreSnapshot` para seguir la evolución diaria.

13. Métricas de la bitácora

En `lib/calculations.ts` se calculan, para operaciones cerradas:

$$\text{WinRate} = \frac{\# \text{ganadoras}}{\# \text{cerradas}} \times 100, \quad (23)$$

$$\text{Expectancy} = \frac{\text{WinRate}}{100} \overline{\text{ganancia}} + \frac{\text{LossRate}}{100} \overline{\text{pérdida}}, \quad (24)$$

$$\text{Sharpe}_{\text{proxy}} = \frac{\overline{P\&L}}{\sigma_{P\&L}} \sqrt{\frac{252}{\text{duración}}}, \quad (25)$$

$$\text{Sortino}_{\text{proxy}} = \frac{\overline{P\&L}}{\sigma_{\text{downside}}} \sqrt{\frac{252}{\text{duración}}}. \quad (26)$$

Además calcula intervalos de confianza al 95 % por *bootstrap* (generador congruencial lineal con semilla determinista) y un *backtest por bandas de puntaje* (< 60 , $60\text{--}74$, $75\text{--}84$, ≥ 85) para comparar el P&L por banda y validar si el modelo discrimina.

14. Bot de investigación en Python

`options-strategy-bot/` es un banco de pruebas independiente que **no opera en vivo**. Replica la lógica de RISKGATE para escanear, puntuar, hacer *backtest* y llevar un libro de *paper trading*.

Módulo	Función
<code>config.py</code>	Esquema de configuración (riesgo, estrategia, costos, universo).
<code>data.py</code>	Carga de datos: CSV, Yahoo o barras sintéticas; modo <i>offline</i> .
<code>features.py</code>	Indicadores (SMA, RSI, volatilidad realizada) y régimen de mercado.
<code>options.py</code>	Black–Scholes propio (precio y Greeks), proxy de IV y construcción de spreads de débito.
<code>model.py</code>	Puntaje de cuatro módulos, decisión y dimensionamiento estilo RISKGATE.
<code>strategies.py</code>	Genera candidatos por estrategia habilitada y los ordena por puntaje.
<code>backtest.py</code>	Backtest histórico con entradas/salidas, curva de equity y resumen (expectancy, win rate).
<code>paper.py</code>	Libro de paper trading en JSON.
<code>providers/alpaca.py</code>	Snapshot de cadena de opciones (solo lectura).
<code>cli.py</code>	Comandos <code>scan</code> , <code>backtest</code> , <code>paper-step</code> .

14.1. Black–Scholes y proxy de IV

El bot valúa opciones con la forma cerrada de Black–Scholes (precio, delta, gamma, theta, vega) y aproxima la IV a partir de la volatilidad realizada a 20 días ($\sigma_{\text{impl}} \approx 1.15 \sigma_{\text{real}}$, acotada a $[0.18, 0.85]$). Esto le permite simular el valor de estructuras sin acceso a cadenas históricas reales.

14.2. Scripts de validación

- `backtest_riskgate_premiums.py`: lee contratos guardados en la base de RISKGATE y traza cómo se movió la prima real (Alpaca) tras la entrada.
- `compare_scored_ideas.py`: compara la precisión direccional y por bandas de puntaje de las ideas guardadas.
- `chart_yahoo_signal_followthrough.py`: grafica el movimiento del subyacente tras una señal usando cierres ajustados de Yahoo.

15. Limitaciones actuales

- El régimen de mercado todavía puede depender de selección manual y usa la tendencia de SPY como *proxy* de régimen para el multiplicador.
- La probabilidad de ganancia y el tope de Kelly dependen de IV y datos de contrato; cuando faltan, se usan *proxies* (probabilidad implícita por puntaje).
- Yahoo Finance es una fuente pública no oficial; puede fallar, retrasarse o cambiar de formato.
- No hay detección automática de fechas de reportes (earnings) ni de eventos macro programados.
- El sentimiento de Reddit usa TF-IDF local, no un modelo de lenguaje, por lo que capta coincidencia de términos más que matices.
- El backtest no modela el deslizamiento real de ejecución ni el llenado parcial de spreads ilíquidos con fidelidad de mercado.
- No hay autenticación multiusuario ni cifrado de la base local.

16. Recomendaciones y próximos pasos

Esta sección reúne las mejoras propuestas, organizadas por horizonte de implementación. Se incluye explícitamente la mejora de los *proxies* que el modelo usa hoy.

16.1. Corto plazo: robustez de datos y *proxies*

Reemplazar el proxy de régimen de SPY. Hoy el multiplicador de régimen G usa únicamente la tendencia de SPY como proxy. Se recomienda construir un *régimen compuesto* que combine SPY, QQQ, breadth y VIX en un solo índice $\in [-1, 1]$ y derivar G de él, de modo que un mercado con SPY plano pero breadth débil y VIX en alza no reciba $G = 0.5$.

Sustituir la probabilidad implícita por puntaje en Kelly. Cuando faltan IV o datos del contrato, el tope de Kelly usa una probabilidad derivada del puntaje ($p = S/100$). Conviene migrar a una PoP calculada siempre desde la cadena real (delta como proxy de probabilidad o PoP lognormal completa), y registrar en la idea si la PoP fue *medida* o *estimada*.

Mejorar el proxy de IV del bot. El factor fijo $1.15 \times \sigma_{\text{real}}$ podría calibrarse por subyacente y por régimen (term structure y *skew* simplificados), o sustituirse por IV real cuando exista snapshot de Alpaca.

Endurecer la fuente de datos. Añadir reintentos, validación de esquema y *fallback* entre proveedores (Yahoo $\rightarrow$ Alpaca $\rightarrow$ manual) con marcas de frescura visibles en la interfaz, y registrar cuándo un dato es estimado.

16.2. Mediano plazo: cobertura del modelo

- **Calendario de earnings y eventos macro:** marcar ideas cuyo vencimiento cruce un reporte o un evento (FOMC, CPI) y aplicar un tope o advertencia automática.
- **Checks por estrategia:** validaciones específicas para *long calls/puts*, *debit spreads*, *protective puts* y *collars* (ancho, break-even, reward/risk, cobertura).
- **Correlación entre ideas:** extender la concentración por sector a una matriz de correlación entre subyacentes para evitar riesgo redundante.
- **Calibración del modelo con la bitácora:** usar el backtest por bandas de puntaje para re-ajustar pesos y umbrales (validación walk-forward).
- **Sentimiento con embeddings semánticos:** reemplazar TF-IDF por *embeddings* de un modelo de lenguaje local para captar matices y sarcasmo, manteniendo el procesamiento en la máquina.

16.3. Largo plazo: producto y plataforma

- **Empaquetado de escritorio:** envolver la app con Tauri para generar un ejecutable de Windows ligero, manteniendo la primera versión de escritorio en modo solo lectura y sin credenciales del broker.
- **Cifrado de la base local** y bloqueo por contraseña local.
- **Auditoría:** bitácora inmutable de decisiones del modelo para revisar por qué se aprobó o rechazó cada idea.
- **Integración de datos profesional:** migrar de Yahoo a un proveedor formal de cadenas y Greeks.

16.4. Candidatos de integración externa

Proveedor	Uso recomendado
Tradier	Cadenas de opciones con Greeks/IV, datos de cuenta y sandbox; ruta hacia paper/live.
Polygon.io	Datos de mercado de opciones de EE. UU. (tiempo real e histórico).
MarketData.app	Vencimientos, strikes, cadenas y cotizaciones para prellenado.
Alpaca	Cadena de opciones ya integrada; paper trading futuro con confirmación explícita.

- Tradier: <https://docs.tradier.com/reference/brokerage-api-markets-get-options-chains>
- Polygon.io: <https://polygon.io/docs/rest/options/overview>
- MarketData.app: <https://www.marketdata.app/docs/api/>
- Alpaca: <https://docs.alpaca.markets/docs/options-trading>

La ruta recomendada es siempre **datos de solo lectura primero**. Cualquier automatización de órdenes debe quedar fuera de alcance hasta que existan manejo confiable de datos, pruebas, auditoría y flujos de confirmación explícita.

17. Cómo ejecutar

17.1. Aplicación web

```
npm install
npm run prisma:generate
npm run db:init
npm run seed
npm run dev
```

Abrir `http://127.0.0.1:3000`. Para datos de opciones, configurar `APCA_API_KEY_ID` y `APCA_API_SECRET_KEY` en `.env`.

17.2. Bot de investigación

```
cd options-strategy-bot
pip install -r requirements.txt
python -m bot.cli scan --synthetic --offline
python -m bot.cli backtest --synthetic
```

18. Notas de seguridad

- Las opciones son riesgosas y pueden expirar sin valor.
- RISKGATE es una herramienta de apoyo a la decisión, no asesoría financiera.
- La aplicación no debe almacenar contraseñas del broker.
- Las integraciones de solo lectura deben preceder a cualquier flujo de colocación de órdenes.
- El modelo es cuantitativo y basado en reglas, no un sistema predictivo garantizado.

19. Conclusión

RISKGATE reúne los componentes de una disciplina de opciones rigurosa: aislamiento por portafolio, posiciones manuales e importadas, watchlist, descubrimiento de candidatos, una compuerta formal de ideas con puntaje ponderado adaptativo, calidad de opciones continua, analítica probabilística (PoP, EV, Kelly, VaR/CVaR), simulación Monte Carlo, sentimiento opcional de Reddit, control de concentración, dimensionamiento acotado por la cuenta y una bitácora con métricas de validación. El bot de Python complementa la app como banco de pruebas de *backtesting*. La evolución más valiosa es continuar **automatizando inputs objetivos y mejorando los proxies del modelo** mientras se conserva la aprobación manual y el principio local-first.